Verification-Gated Local Model Cascading

An EvalPlus Reproducibility Study of Task-Level Complementarity

Empirical Research Article

Neeraj Pragnya Krishna Vasagiri

Independent Researcher

Version 1.1 – August 2026

Corrected preprint – Not peer reviewed

RESEARCH STATUS

This article reports one completed local benchmark experiment on 542 EvalPlus tasks. The results support task-level model complementarity and verification-gated escalation in this tested setting; they do not establish universal efficiency, general superiority, or improved model intelligence.

AI assistance disclosure: AI systems, including ChatGPT and Codex, assisted with code inspection, statistical checking, organization, editing, formatting, and audit support. The author reviewed the evidence and accepts responsibility for the manuscript.

Abstract

Large language model failures need not be nested: a smaller model can sometimes pass a coding benchmark task that a larger model fails. This paper studies a sequential local cascade in one completed experiment on 542 EvalPlus tasks (164 HumanEval+ and 378 MBPP+) using Qwen2.5-Coder-1.5B-Instruct, 3B-Instruct, and 7B-Instruct Q4_K_M tiers. Adaptive produced 331 EvalPlus Plus test passes (61.07%), Strong-only 297 (54.80%), and Small-only 220 (40.59%). Its 34 additional passes over Strong-only came from FAST (22) and BALANCED (12); BEST contributed none. The same EvalPlus Plus tests controlled Adaptive acceptance or escalation and defined the primary endpoint. This is therefore an oracle-gated complementarity study, not evidence for an independently deployable verifier. Adaptive also used 1,092 model calls versus 542 for Strong-only, 121.04% more raw tokens, 14.11% more sample-integrated CPU-seconds, and 65.40% more summed generation-and-verification wall time. Model-size-weighted token quantities are accounting proxies, not FLOPs, hardware compute, energy, API usage, or dollar cost. The experiment does not isolate predictive routing, repeated-attempt effects, or universal efficiency.

Keywords: code generation, model cascading, oracle-gated evaluation, LLM routing, EvalPlus, local inference, reproducibility

1 Introduction

Model correctness sets can be non-nested: one model may solve tasks another misses. A cascade can exploit this complementarity by trying another tier after a failed attempt. A rigorous interpretation must distinguish complementarity from the advantage of receiving additional attempts and must separate a deployment-time verifier from the final evaluation criterion.

The historical Adaptive Workflow benchmark generated one candidate at a local tier and evaluated it with the pinned EvalPlus Plus suite. A Plus-test pass terminated the cascade; failure permitted escalation. Later tiers received the original frozen prompt, not failed code or test feedback. Because the same Plus result controlled escalation and final scoring, the study is oracle-gated and does not test a separately trained or independently held-out verifier.

On the same 542 tasks, Adaptive passed 331, Strong-only 297, and Small-only 220. The 34 Adaptive-only passes were produced by FAST (22) and BALANCED (12). Since Adaptive can retain earlier-tier passes and later fall back to the same BEST tier as Strong-only, zero Strong-only-only passes is structurally unsurprising under deterministic-equivalent common attempts. The central finding is non-nested model-tier pass sets under oracle gating.

1.1 Research questions

RQ1. Under an EvalPlus-oracle gate, how many additional Plus-test passes does a multi-tier cascade obtain relative to one-attempt baselines?
RQ2. How much of the observed gain reflects non-nested task-level pass sets among differently sized tiers?
RQ3. What additional raw-system expenditure occurs under tokens, model-size-weighted accounting proxies, sampled CPU/RAM telemetry, model calls, and summed generation-and-verification time?
RQ4. What evidence, if any, supports predictive initial routing beyond a simple FAST-first oracle-gated cascade?

2 Related Work

FrugalGPT studies cascades under API-cost constraints [2]; RouteLLM learns a preference-data router [11]; and early-abstention work studies avoiding expensive calls [15]. A unified routing/cascading treatment [5] and BEST-Route's joint model/sample allocation [6] show why fixed-cascade, repeated-sampling, and held-out-routing baselines matter. Faster cascades and speculative decoding operate at a different, token-level mechanism [10, 8].

CALM allocates effort using calibrated early exits [12], while test-time allocation work uses explicit matched comparisons [13]. Self-Debugging passes execution feedback into later generations [4]; this historical cascade did not. Self-consistency samples multiple paths [14]; this study used one deterministic completion per tier.

HumanEval and MBPP are public function-synthesis benchmarks [3, 1]; EvalPlus strengthens their tests [9]. Passing EvalPlus is still a finite test-suite outcome, not proof of semantic correctness. Oracle or test-contract

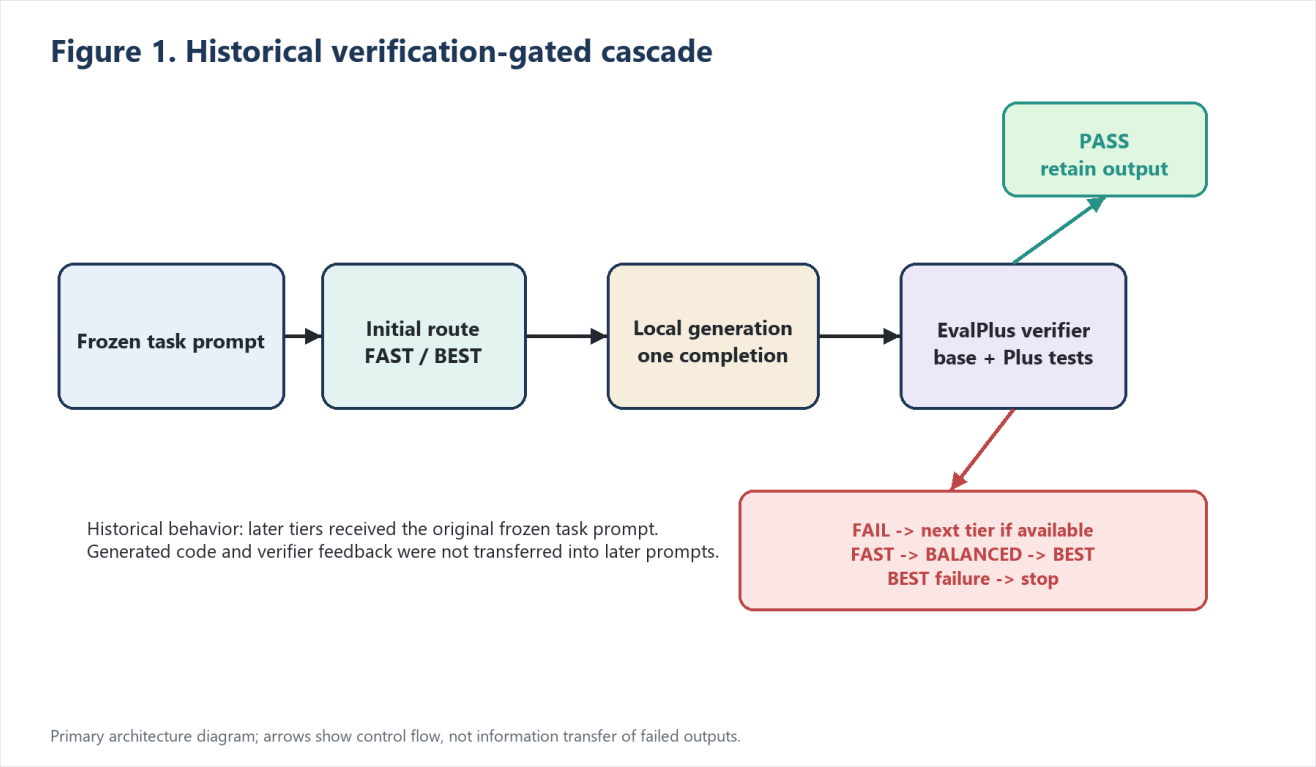


Figure 1: Historical EvalPlus-oracle-gated cascade. Later tiers received the original frozen prompt, not failed code or test feedback.

defects can cause false acceptance or rejection. LiveCodeBench and BigCodeBench provide newer contamination-aware and broader coding context [7, 16]; no external leaderboard comparison is made here.

3 Adaptive Workflow

For a task x, the system selects a tier, obtains a completion y, and executes the pinned EvalPlus evaluator $V(x, y)$. A Plus-test pass retains y; otherwise the historical policy may advance FAST $\rightarrow$ BALANCED $\rightarrow$ BEST. Infrastructure failures are recorded and are not retried into a pass. Because V also defines the primary endpoint, it is an oracle benchmark gate rather than an independent deployment verifier.

The tiers were Qwen2.5-Coder-1.5B-Instruct, 3B-Instruct, and 7B-Instruct, all Q4_K_M, with frozen accounting values near 1.5B, 3.1B, and 7.6B. The 1.5B and 7B upstream models are Apache-2.0; the 3B model uses the Qwen Research License. The router used hand-written task features plus online RoutingStatistics, not a trained predictive model. Its confidence field was a fixed implementation value, not a calibrated probability. Initial routes were FAST 531, BALANCED 0, and BEST 11.

4 Experimental Method

The fixed task set contains 164 HumanEval+ and 378 MBPP+ tasks. The dataset repository pins were:

HumanEval	`6d43fb980f9fee3c892a914eda09951f772ad10d`
MBPP	`f46ca8374b4cddef97ca4208ad986049d74d296a`

EvalPlus was pinned to v0.3.1; HumanEval+ to v0.1.10; and MBPP+ to v0.2.0. Base and Plus are two test views of the same completion, not independent observations. Small-only made one FAST attempt, Strong-only one BEST attempt, and Adaptive at most one attempt per tier. No historical Balanced-only, fixed-cascade, repeated-BEST, repeated-FAST, or budget-matched repeated-sampling baseline was run.

Table 1: Frozen benchmark configuration.

Item	Frozen value	Size/count	Notes
HumanEval+	v0.1.10	164	EvalPlus augmented tests
MBPP+	v0.2.0	378	EvalPlus augmented tests
Total	same paired task set	542	one completion scored on base and Plus
FAST	qwen2.5-coder:1.5b	1.5B	Q4_K_M
BALANCED	qwen2.5-coder:3b	3.1B	Q4_K_M
BEST	qwen2.5-coder:7b	7.6B	Q4_K_M
Sampling	temperature 0; top_p 1	seed 20260824	one thread; 1024 output cap
Oracle gate / endpoint	EvalPlus	v0.3.1	Docker network none; 120 s timeout
Resource ceiling	CPU 45% soft / 50% hard	RAM 45% soft / 50% hard	process-tree controls

Table 2: Primary EvalPlus Plus test pass rates. Overall is task-weighted.

Condition	HumanEval+	MBPP+	Overall	Macro average
Small-only	26/164 (15.85%)	194/378 (51.32%)	220/542 (40.59%)	33.59%
Strong-only	34/164 (20.73%)	263/378 (69.58%)	297/542 (54.80%)	45.15%
Adaptive	43/164 (26.22%)	288/378 (76.19%)	331/542 (61.07%)	51.20%

Temperature was 0, top-p 1, seed 20260824, context window 8,192, output cap 1,024, and one model thread. CPU and RAM were sampled every 0.5 seconds. Reported CPU-seconds are sample-integrated estimates, not direct OS cumulative CPU accounting; peaks may miss sub-sample spikes. Reported wall time is summed generation-and-verification time, not total experiment elapsed time.

For tier parameter value P in billions, $C_{\text{generation}} = P \times \text{output_tokens}$ and $C_{\text{total}} = P \times (\text{input_tokens} + \text{output_tokens})$. These are model-size-weighted token accounting proxies, not measured FLOPs, hardware work, energy, API usage, API credits, dollar cost, or generic resource consumption. The frozen budget baseline was $B_0 = 4,818,407.6$ with 25/50/75/100% cutoffs. Every condition exhausted its task list before the first cutoff, so the frozen curve is nondiscriminating and provides no equal-budget evidence.

The router updated statistics online using earlier EvalPlus outcomes, so later test-set decisions could depend on earlier labels and on the one frozen task order. This is an online sequential evaluation, not a held-out frozen-router evaluation. Tier histories are conditionally selected: FAST sees nearly all tasks, BALANCED sees FAST failures, and BEST sees a harder residual subset.

5 Results

Adaptive passed 331/542 (61.07%), Strong-only 297/542 (54.80%), and Small-only 220/542 (40.59%). Adaptive's difference from Strong-only is 34 tasks, 6.27 percentage points, or 11.45% more successful tasks on this fixed workload. Paired outcomes were 297 both pass, 34 Adaptive only, 0 Strong-only only, and 211 both fail. The exact McNemar result $p = 1.164 \times 10^{-10}$ is post-hoc, secondary, and architecturally asymmetric; the effect count and percentage-point difference are the primary description.

The 34 Adaptive-only passes comprise 22 FAST and 12 BALANCED outputs; BEST contributes zero. Final Adaptive pass tiers were FAST 216, BALANCED 69, and BEST 46. Thus 285/331 (86.10%) of successful Adaptive tasks did not invoke BEST. This is tier-use composition, not an efficiency metric. Conditional tier rows are not standalone 542-task capability estimates.

Adaptive used 166,716 input, 93,006 output, and 259,722 total tokens versus 78,993, 38,507, and 117,500 for Strong-only. Weighted input, output, and total proxies were 579,460.7, 320,002.1, and 899,462.8 versus 600,346.8, 292,653.2, and 893,000.0. The weighted-total difference is +0.72%. Adaptive also used 1,092 model calls and 1,091 verifier calls versus 542 and 542; sample-integrated CPU-seconds 38,137.61 versus 33,421.46 (+14.11%); and summed generation-and-verification wall seconds 78,701.93 versus 47,582.32 (+65.40%).

Six tasks produced shared verifier/infrastructure failures in all conditions: Mbpp/106, Mbpp/124, Mbpp/252, Mbpp/255, Mbpp/580, and Mbpp/720. They remain in the 542-task denominator. Preserved errors and static source inspection strongly indicate that raw MBPP+ JSON reached ground-truth execution without EvalPlus input deserialization for five tasks (106, 124, 252, 580, and 720), causing task-specific type errors. For Mbpp/255, ground-truth computation preceded a cache-pickle write that failed with no space left; static evidence is consistent with the verifier's documented 512 MiB temporary filesystem being the binding limit on an exceptionally large

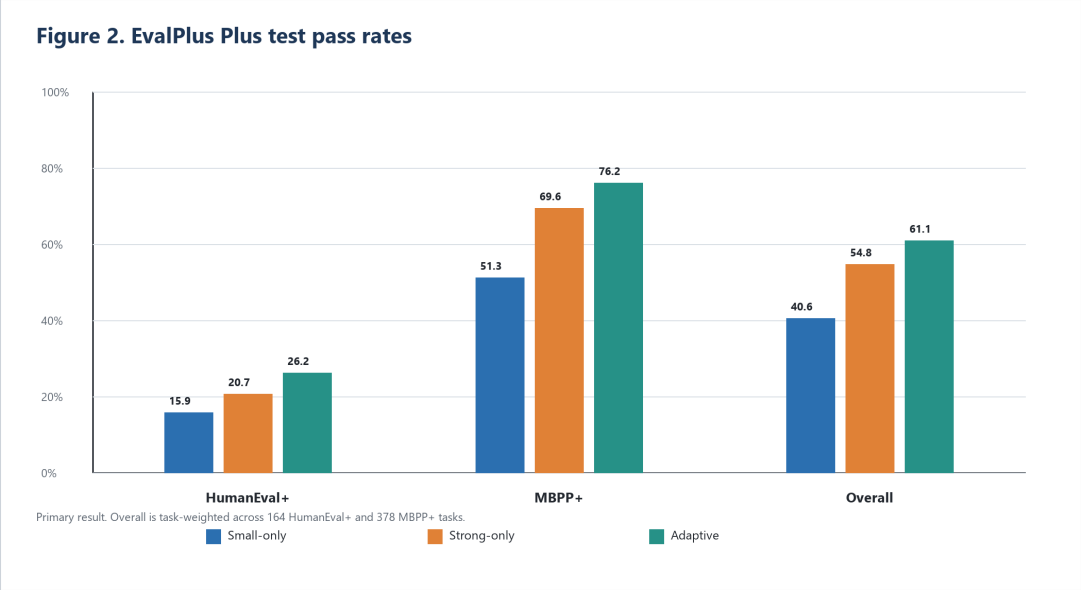


Figure 2: Primary EvalPlus Plus test pass rates.

Table 3: Adaptive versus Strong-only expenditure and sampled telemetry.

Metric	Adaptive	Strong-only	Relative difference
Plus successes	331	297	+11.45%
Raw input tokens	166,716	78,993	+111.05%
Raw output tokens	93,006	38,507	+141.53%
Raw total tokens	259,722	117,500	+121.04%
Weighted input proxy	579,460.70	600,346.80	-3.48%
Weighted output proxy	320,002.10	292,653.20	+9.35%
Weighted total proxy	899,462.80	893,000	+0.72%
Sample-integrated CPU-seconds estimate	38,137.61	33,421.46	+14.11%
Summed generation-and-verification wall seconds	78,701.93	47,582.32	+65.40%
Peak CPU	47.12	46.89	+0.235 pp
Peak RAM bytes	6,897,147,346	6,999,388,192	-1.46%
Model calls	1,092	542	+101.48%
Verifier calls	1,091	542	+101.29%

Plus input. The failure symptoms are directly preserved; the causal mapping is a strong inference because the exact historical source tree is unavailable. These findings explain rejection paths but do not supply corrected historical candidate outcomes.

6 Analysis

The static FAST-versus-BEST comparison on all 542 tasks has 22 FAST-only passes, 99 BEST-only passes, 198 both-pass, and 223 both-fail outcomes, directly showing non-nested pass sets. Adaptive's retained 34 additional passes are consistent with that complementarity. Adaptive also received more oracle-checked attempts, so this design does not isolate complementarity from a general more-attempts effect.

Initial routing was 97.97% FAST-first. Combined with the uncalibrated fixed confidence value, online label updates, task-order dependence, and conditional tier exposure, the run is weak evidence for predictive initial routing. It is stronger evidence about oracle-gated cascading and tier complementarity.

Adaptive's post-hoc successes-per-weighted-total-proxy ratio is 10.65% higher, but this secondary proxy is neither equal-budget evidence nor a physical efficiency measure. Raw tokens, model calls, verifier calls, sampled CPU time, and summed generation-and-verification time are higher. The result is a correctness/cost trade-off under one operating point, not an overall efficiency advantage.

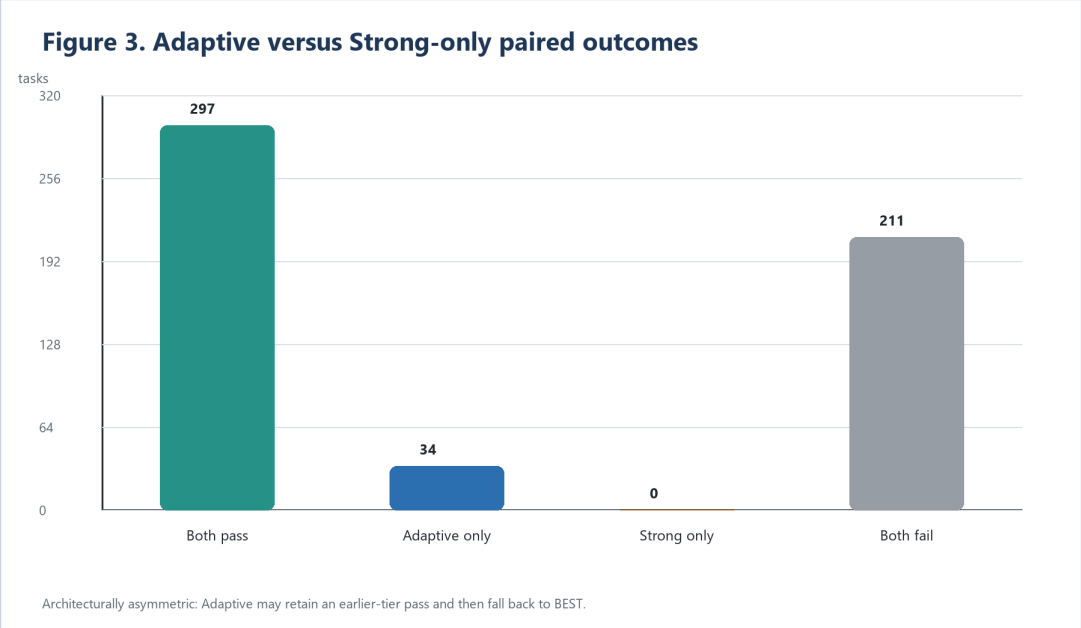


Figure 3: Paired Adaptive and Strong-only outcomes. Design asymmetry is explicit.

7 Limitations

The same EvalPlus Plus outcome controls escalation and final scoring; this is evaluator leakage by design. Adaptive can retain earlier passes and fall back to BEST, while Strong-only receives one BEST attempt. Missing baselines include Balanced-only, fixed cascade, repeated-BEST, repeated-FAST, and budget-matched alternatives. Online routing uses earlier test labels, has conditional selection bias, and was evaluated under one task order.

This is one deterministic run on one Windows machine, one model family, one quantization family, one runtime configuration, and one seed. HumanEval and MBPP are old public benchmarks with plausible contamination and do not measure repository-scale software engineering or general modern coding ability. EvalPlus is a finite instrument with possible oracle/test defects. Six shared failures remain in the primary denominator; their static infrastructure causes are documented, but corrected outcomes require a new run. Exact historical generated programs were not retained, preventing output-level re-evaluation. The current public harness includes post-run improvements and is not byte-identical to the historical source identified by its preserved hash.

8 Future Work

Future work should separate deployment-time verification from held-out final evaluation; add Balanced-only, fixed cascade, repeated-FAST, repeated-BEST, and budget-matched alternatives; use a separately calibrated frozen router, multiple seeds and task orders, multiple model families and quantizations; evaluate LiveCodeBench and BigCodeBench; validate the corrected MBPP adapter in a new preregistered run; and retain exact private generated outputs and hashes.

9 Conclusion

On this fixed 542-task workload, the oracle-gated cascade produced 331 Plus-test passes versus 297 for one-attempt Strong-only and 220 for Small-only. The 34 Adaptive-only passes came entirely from FAST and BALANCED. This demonstrates non-nested pass sets among the tested tiers under the pinned oracle gate. It does not establish an independently deployable verifier, predictive-routing superiority, equal-budget superiority, or universal efficiency.

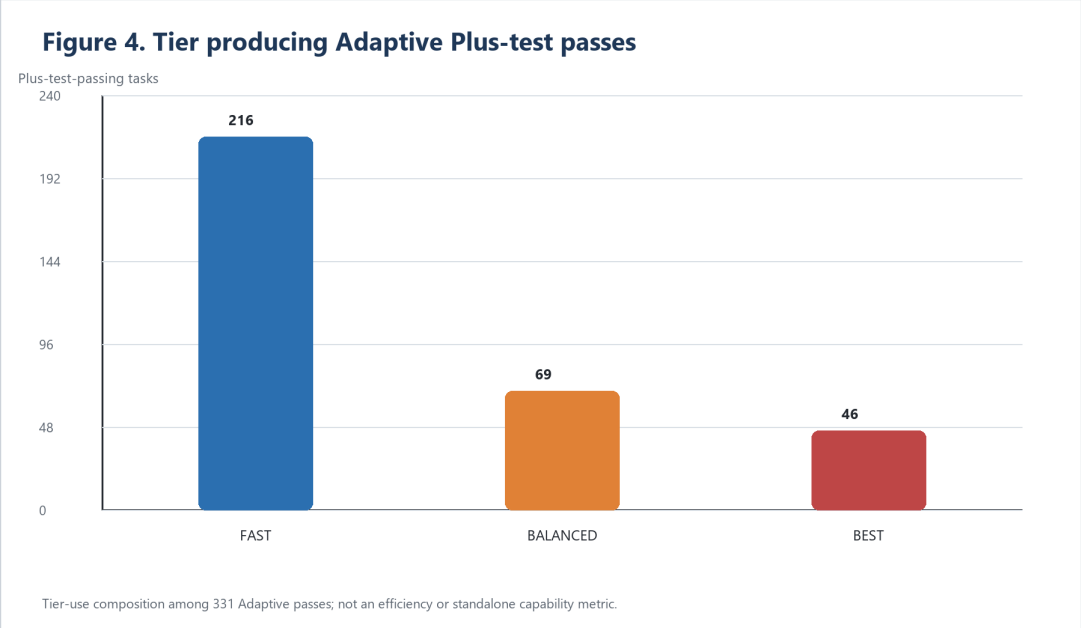


Figure 4: Source tier for final Adaptive Plus-test passes; this is tier-use composition, not an efficiency measure.

AI assistance disclosure

AI systems, including ChatGPT and Codex, assisted with code inspection, statistical checking, scientific critique, organization, drafting, editing, formatting, and audit support. The author originated the research direction and architecture, reviewed the evidence and analyses, made the final methodological and interpretive decisions, approved the manuscript revisions, and accepts responsibility for all claims. AI assistance is not presented as independent peer review.

Reproducibility and provenance

The public repository contains the paper, analysis code, sanitized results, provenance, harness snapshot, and scientific audit. Historical exact generated text was not retained. The unchanged hashes are:

Raw JSONL SHA-256	`6926147073d0e5b3ffe93ecb3d9c8619707b24f0b48a843d25db6627602268da`
SQLite SHA-256	`138d04bc1c2f3b6609a56cc2a5e77a6d024035f22d948f81a674139ef06976e7`
Frozen source-tree SHA-256	`0d24437e9297466c3f28a7af677a76d7e54c10dc6358edb7b724d586040e7859`

The v1.0.0 annotated tag truncated the SQLite hash in its tag message; the tag remains unchanged as historical provenance.

License declaration

© 2026 Neeraj Pragnya Krishna Vasagiri. Paper text and original figures and tables are licensed under CC BY 4.0. Project software is MIT unless otherwise noted. HumanEval, MBPP, EvalPlus, Qwen model artifacts, and other third-party materials retain their upstream licenses; nothing here relicenses third-party material.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Xinyun Chen, Maxwell Lin, Nathanael Scharli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.

[5] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A unified approach to routing and cascading for llms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 12987–13010. PMLR, 2025.

[6] Dujian Ding, Ankur Mallick, Shaokun Zhang, Chi Wang, Daniel Madrigal, Mirian Del Carmen Hipolito Garcia, Menglin Xia, Laks V. S. Lakshmanan, Qingyun Wu, and Victor Rühle. BEST-route: Adaptive LLM routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 13870–13884. PMLR, 2025.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[9] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36*, 2023. EvalPlus; official proceedings record.

[10] Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Seungyeon Kim, Neha Gupta, Aditya Krishna Menon, and Sanjiv Kumar. Faster cascades via speculative decoding. In *International Conference on Learning Representations*, 2025.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data. In *International Conference on Learning Representations*, 2025.

[12] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems 35*, 2022.

[13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*, 2025.

[14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[15] Michael J. Zellinger, Rex Liu, and Matt Thomson. Cost-saving llm cascades with early abstention. *arXiv preprint arXiv:2502.09054*, 2025.

[16] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

A Audit details

The audit reports 2,176 attempts from 1,626 completed condition-task cells and zero duplicate attempt keys. Exact historical candidate text was not retained, so deterministic-equivalence findings rely on observable metadata, outcomes, tokens, frozen generation options, and model digests rather than byte-for-byte output comparison.